



LeetCode 69: Sqrt(x)

1. Problem Title & Link

Title: LeetCode 69: Sqrt(x)

Link: <https://leetcode.com/problems/sqrtx/>

2. Problem Statement (Short Summary)

Given a non-negative integer x , return the **integer square root** of x .

The integer square root is:

$\text{floor}(\sqrt{x})$

meaning the largest integer whose square is less than or equal to x .

You are **not allowed** to use built-in square root functions.

Example

Input:

$x = 8$

Actual square root:

$\sqrt{8} = 2.828\dots$

Integer square root:

2

Output:

2

3. Examples (Input → Output)

Example 1

Input:

$x = 4$

Output:

2

Explanation:

$2 \times 2 = 4$

Example 2

Input:

$x = 8$

Output:

2

Explanation:



$\sqrt{8} = 2.828...$

Floor value:

2

Example 3

Input:

$x = 16$

Output:

4

Example 4

Input:

$x = 27$

Output:

5

Explanation:

$5^2 = 25$

$6^2 = 36$

Answer:

5

4. Constraints

$0 \leq x \leq 2^{31} - 1$

Important:

Do NOT use `sqrt()`.

Do NOT use `pow()`.

5. Core Concept (Pattern / Topic)

Binary Search

Secondary Topics:

- Mathematics
- Search Space Reduction

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

Check every number.



Example:

$$x = 16$$

Try:

$$1^2 = 1$$

$$2^2 = 4$$

$$3^2 = 9$$

$$4^2 = 16 \checkmark$$

Answer:

4

Problem

For:

$$x = 2147395599$$

checking every number becomes very slow.

Key Observation

Square roots form a sorted sequence.

$$1^2 = 1$$

$$2^2 = 4$$

$$3^2 = 9$$

$$4^2 = 16$$

$$5^2 = 25$$

$$6^2 = 36$$

Squares increase monotonically.

This means:

Binary Search can be used.

Search Space

For:



$x = 16$

Possible answers lie between:

0 and 16

Suppose:

$mid = 8$

Check:

$8^2 = 64$

Too large.

Answer must be on the left.

Suppose:

$mid = 4$

Check:

$4^2 = 16$

Found answer.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

$x = 8$

Search Space:

0 ----- 8

Iteration 1

$mid = 4$

$4^2 = 16$

Too large.

Move left.

0 ---- 3

Iteration 2

$mid = 1$

$1^2 = 1$

Too small.

Move right.

Iteration 3

$mid = 2$



```
22 = 4
Still small.
Store answer:
2
Move right.
```

```
Iteration 4
mid = 3
```

```
32 = 9
Too large.
Stop.
Answer:
2
```

8. Pseudocode (Language Independent)

```
left = 0
right = x

answer = 0

while left <= right

    mid = (left + right) / 2

    square = mid * mid

    if square == x

        return mid

    if square < x

        answer = mid

        left = mid + 1

    else

        right = mid - 1

return answer
```



9. Code Implementation

Python

```
class Solution:
    def mySqrt(self, x: int) -> int:

        left = 0
        right = x

        answer = 0

        while left <= right:

            mid = (left + right) // 2

            square = mid * mid

            if square == x:
                return mid

            elif square < x:

                answer = mid
                left = mid + 1

            else:
                right = mid - 1

        return answer
```

Java

```
class Solution {

    public int mySqrt(int x) {

        int left = 0;
        int right = x;

        int answer = 0;

        while(left <= right) {
```



```
int mid = left + (right - left) / 2;

long square = (long) mid * mid;

if(square == x)
    return mid;

else if(square < x) {

    answer = mid;
    left = mid + 1;

}

else {
    right = mid - 1;
}

return answer;
}
```

10. Time & Space Complexity

Time Complexity

$O(\log x)$

Reason:

Binary Search halves the search space every iteration.

Space Complexity

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

`sqrt()`

Not allowed.

Mistake 2

Overflow.



Example:

$\text{mid} * \text{mid}$

can exceed integer range.

Java solution uses:

long

Mistake 3

Returning:

left

instead of the last valid answer.

Need:

answer

Edge Cases

Input	Output
0	0
1	1
4	2
8	2
15	3
16	4

12. Detailed Dry Run

Input:

$x = 8$

Initial:

left = 0

right = 8

answer = 0

left	right	mid	mid ²	Action
0	8	4	16	Too Large
0	3	1	1	Store 1
2	3	2	4	Store 2
3	3	3	9	Too Large

Loop ends.

Answer:



2

13. Common Use Cases (Real-Life / Interview)

Binary Search on Answers

This problem introduces an important interview technique:

Search the Answer Space

Engineering Calculations

Square-root approximations.

Scientific Computing

Numerical methods.

Competitive Programming

Foundation for advanced binary search problems.

14. Common Traps (Important!)

Trap 1

Thinking Binary Search works only on arrays.

Actually:

Binary Search works on any sorted search space.

Trap 2

Overflow in:

$mid * mid$

Trap 3

Returning wrong boundary.

Need the largest value satisfying:

$mid^2 \leq x$

15. Builds To (Related LeetCode Problems)

LC 69 - Sqrt(x)

↓

LC 367 - Valid Perfect Square

↓

LC 278 - First Bad Version



LC 875 - Koko Eating Bananas



LC 1011 - Capacity To Ship Packages

These are classic Binary Search-on-Answer problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Linear Search	$O(\sqrt{x})$	$O(1)$	Poor
Binary Search	$O(\log x)$	$O(1)$	Best
Newton Method	$O(\log x)$	$O(1)$	Advanced

Approach 1: Linear Search

Try:

$$1^2$$

$$2^2$$

$$3^2$$

...

Simple but slow.

Approach 2: Binary Search (Chosen)

Most commonly expected.

Approach 3: Newton's Method

Advanced mathematical optimization.

Used in scientific computing.

17. Why This Solution Works (Short Intuition)

The square values increase in sorted order.

If:

$$\text{mid}^2 > x$$

the answer must lie to the left.

If:

$$\text{mid}^2 < x$$

the answer must lie to the right.

This monotonic property makes Binary Search the ideal solution.



18. Variations / Follow-Up Questions

Follow-Up 1

Check whether a number is a perfect square.

Related:

LC 367

Follow-Up 2

Find cube root.

Example:

27

Answer:

3

Follow-Up 3

Return decimal precision.

Example:

$\sqrt{8} = 2.8284...$

up to 3 decimal places.

Follow-Up 4

Find nth root using Binary Search.

Example:

$\sqrt[4]{81}$

Answer:

3

Interview Takeaway

This is one of the most important Binary Search problems because it teaches:

Binary Search on an Answer

Whenever you see:

Find maximum value

Find minimum value

Find closest value

and the search space is ordered, think:

Binary Search

Even if there is no array involved.

